Product Requirements Document (PRD)

Real-Time E-Commerce Data Platform with Cloud Lakehouse and Data Observability

1. Project overview and problem statement

E-commerce data arrives from customers, products, orders, payments, inventory, and order events. Duplicates, missing records, invalid values, and inconsistent statuses can make reporting unreliable. The platform preserves raw data, validates and transforms records, and publishes analytics-ready datasets.

2. Goals and success criteria

Ingest CSV/JSON and simulated events; preserve Bronze data; validate Silver data; produce Gold analytics; record quality outcomes and pipeline runs; provide reproducible setup and tests.

3. Scope and non-goals

In scope: batch ingestion, simulated streaming, Bronze/Silver/Gold, quality rules, orchestration, logs, analytics SQL, tests, documentation. Non-goals initially: shopping website, real payments/card data, production-scale infrastructure, ML recommendations, or paid cloud without approval.

4. Functional requirements

FR-01 load synthetic files; FR-02 capture source metadata; FR-03 store Bronze; FR-04 validate Silver; FR-05 handle duplicates; FR-06 create Gold; FR-07 log run status/counts/errors; FR-08 persist quality results; FR-09 preserve rejects; FR-10 process simulated events; FR-11 orchestrate tasks; FR-12 monitoring; FR-13 tests and instructions.

5. Non-functional requirements

Reliability, maintainability, observability, security, reproducibility, cost control, and measurable performance. Idempotency must be implemented and tested.

6. Proposed data model

customers, products, orders, order_items, payments, inventory, order_events, pipeline_runs, and quality_results. Final columns, types, keys, and deduplication rules require review.

7. Data flow and layers

Sources -> ingestion -> Bronze raw data -> Silver validated/cleaned data -> Gold business aggregates -> reports. Quality results and run logs span all layers.

8. Processing and orchestration

Batch processes groups of files. Streaming receives events; the first version may simulate it locally. Orchestration manages task dependencies and status. Kafka and Airflow are potential later-stage tools.

9. Quality and observability

Check required IDs, duplicate event IDs, positive quantities, valid statuses, relationships, and payment/order consistency. Track run status, counts, duration, last successful run, failures, and errors.

10. Proposed technology stack

Python, SQL, Git/GitHub, pytest, Parquet, PostgreSQL, and optionally Kafka, Spark, Airflow, Docker, Delta Lake/Iceberg, and Power BI. Confirm cloud choices and costs with the senior.

11. Acceptance criteria

README setup works; layers produce expected outputs; invalid records have reasons; reruns avoid unintended duplicates; failures are logged; Gold metrics reconcile; tests pass.

12. Risks, assumptions, dependencies

Risks: setup complexity, cloud costs, duplicate/late events. Assumptions: synthetic data and portfolio use. Dependencies: Python, Git/GitHub, sample data, approved local tools.

13. Deliverables and milestones

Deliverables include PRD, HLD, diagram, code, sample data, tests, quality report, logs, Gold outputs, instructions, and demo. Milestones: design; repository/data; Bronze; Silver/Gold; streaming/orchestration; observability; final review.

14. Questions and approval

Confirm cloud platform, Kafka/Airflow need, Delta/Iceberg choice, dashboard requirements, deadline, standards, and evaluation criteria. Status: Pending senior review.